

MenaFina: Intelligent Financial Education Platform – Technical Report

1. Abstract

MenaFina is a comprehensive educational and financial platform that integrates learning, entrepreneurial support, investment simulation, and AI-driven assistance. The project addresses the critical issue of limited intuitive, personalized, and risk-managed financial guidance available to beginner investors. Our objective is to democratize finance and investment education, making it accessible and personalized for all users.

A .AI Chatbot – Entrepreneur Profile & Portfolio Diversification :

Classifies the user's entrepreneurial profile (goals, risk tolerance) and generates a tailored investment portfolio. An RL agent then automatically adjusts and optimizes the portfolio over time.

B. Professor–Student Space :

Facilitates educational content via published courses and scheduled webinars, featuring calendar synchronization and a course rating system.

C. Investment Simulator (Machine Learning):

Uses a financial API to retrieve indicators for a user-inputted ticker. A Random Forest model then predicts whether the user should invest or not.

D. User Account Management :

Provides a personal dashboard for role management, notifications, and calendar sync. The system also targets students, professors, and beginner investors seeking smart, data-driven recommendations.

E.Admin Space (Future):

Will provide platform-wide management capabilities, including overseeing users, courses, webinars, and analytics dashboards.

2. Methodology

2.1. Data

Data Sources

The MenaFina platform utilizes multiple data sources to power its intelligent features:

Synthetic User Dataset

- **Purpose:** Training dataset for investor profiling (ML classification model)
- **Content:** Synthetic records representing various investor types (risk level, capital range, investment horizon, financial experience)

- **Format:** Structured tabular dataset
- **Usage:** Model training + validation to classify users into investment profiles
- **Advantage:** Allows balanced profile distribution without using sensitive real user data

Optimizer Market Dataset

- **Purpose:** Dataset dedicated to portfolio optimization (HRP / allocation engine)
- **Content:** Historical data of ~52 selected tickers downloaded from yfinance (from 2015 onward)
- **Format:** Time-series market dataset (Close prices primarily)
- **Usage:** Covariance/correlation estimation, risk-based weighting, portfolio allocation generation
- **Role:** Benchmark and real-data foundation for recommendation outputs

RL Training Dataset

- **Purpose:** Training environment for the Reinforcement Learning agent
- **Content:** ~7 years of historical financial market time-series (prices, returns, market variations)
- **Format:** Sequential financial time-series retrieved via public finance API (yfinance)
- **Usage:** Simulates market behavior for RL interaction → reward calculation & policy optimization
- **Output:** Learned allocation strategy/model weights exploitable by the chatbot

User Saved Data (JSON Storage)

- **Purpose:** Stores user interactions and investment recommendations generated by the chatbot
- **Format:** JSON file structured by username → history entries
- **Content:** For each session, the file records:
 - date of interaction
 - detected profile (via ML model)
 - invested capital
 - hrp_portfolio allocation (weights %)
 - rl_portfolio allocation (amount or simulation result when available)
- **Usage:**
 - Tracks user investment history and portfolio evolution
 - Allows the chatbot to display previous sessions, comparisons, or evolution over time

Educational Content Data:

- **Source:** Professor-created courses, tutorials, and webinars
- **Format:** Video files (video_url), PDF documents (file_url), metadata (titles, descriptions)
- **Storage:** Supabase with cloud storage links

Data Format and Size

Investment Simulator Dataset: The Finance API returns comprehensive financial indicators for each company query:

Market Data Features:

- Ticker, Date, Close_Adj, High, Low, Open, Volume
- Daily_Return, Volatility, Annual_Volatility
- MA10, MA50, MA200 (Moving Averages)
- RSI (Relative Strength Index), MACD (Moving Average Convergence Divergence)
- Drawdown, Momentum indicators (1D, 5D, 10D, 20D)
- Return_Lag indicators (1D, 5D, 10D)
- Temporal features: Day_of_Week, Month, Quarter

Financial Statement Features:

- Income Statement: Total Revenue, Operating Revenue, EBITDA, EBIT, Operating Income, Net Income, Net Income From Continuing Operations, Net Income Common Stockholders, Diluted/Basic EPS
- Balance Sheet: Total Debt, Net Debt, Current Assets, Current Liabilities, Cash & Equivalents, Accounts Receivable, Inventory, Invested Capital, Common Stock Equity
- Derived Metrics: ROE, Profit_Margin, Debt_to_Equity, EPS_Growth, Revenue_Growth, Current_Ratio, Quick_Ratio, Operating_Margin, P_to_E, Price_to_Book, Price_to_Revenue, Cost Of Revenue

Target Variable:

- Binary classification: Invest (1) or Do Not Invest (0)

Dataset Size: Dynamic and company-specific; typically includes hundreds to thousands of data points per company depending on historical data availability.

Data Preprocessing and Cleaning Steps

For Investment Simulator:

1. **API Data Retrieval:** Fetch raw financial data from Finance API based on user-provided ticker symbol
2. **Missing Value Handling:** Identify and handle missing values in financial indicators (imputation or exclusion based on feature importance)
3. **Feature Engineering:**
 - Calculate technical indicators (RSI, MACD, Moving Averages)
 - Compute momentum and return lag features
 - Extract temporal features (day of week, month, quarter)
4. **Normalization/Standardization:** Scale numerical features to ensure model convergence and performance
5. **Outlier Detection:** Identify and handle extreme values that could skew predictions
6. **Feature Selection:** Select the most relevant features for the Random Forest model based on importance scores

For Chatbot :

1. Synthetic User Dataset

- Numeric encoding of user attributes (risk tolerance, experience level, investment horizon...)
- Normalization/scaling where needed for ML training
- Class balance verification to avoid biased profile prediction
- Removal of unrealistic entries during dataset generation

2. Market Financial Data (RL + Optimizer Dataset)

- Retrieval from API (yfinance)
- NaN and missing date entries removed using .dropna()
- Time-series aligned by date to ensure continuity
- Returns computed from closing prices for reward calculation
- Optional: rolling features (volatility, moving averages) when used

3. Portfolio Optimizer Dataset (~25 tickers since 2015)

- Price series merged into a unified dataframe
- Conversion to percentage returns for HRP covariance matrix calculation
- Outliers smoothing if necessary (extreme spikes)
- Correlation matrix computed for diversification

4. RL Training Preparation

- Time-series segmented into episodes for training environment
- Conversion into state matrix used by the RL agent
- Calculation of reward from portfolio performance at each step
- Normalization to stabilize learning

5. User Session JSON Storage

- Each interaction appended to the JSON file
- Verification structure format before writing
- Null values allowed when RL portfolio is not generated

For Educational Content:

1. **Content Validation:** Ensure uploaded courses/tutorials meet platform standards (file format, size limits)
2. **Metadata Extraction:** Extract and structure course information (title, description, duration)
3. **Quality Control:** Validate video/PDF URLs and accessibility

2.2. Approach / Algorithms

Investment Simulator: Random Forest Model

Model Selection Rationale: Random Forest was chosen for the investment prediction task due to:

- Ability to handle high-dimensional data with numerous financial features

- Robustness to overfitting through ensemble learning
- Capability to capture non-linear relationships in financial data
- Built-in feature importance rankings for interpretability
- Strong performance on structured/tabular data

Key Design Decisions:

Model Architecture:

- Algorithm: Random Forest Classifier
- Input Features: 60+ financial and technical indicators
- Output: Binary classification (Invest: 1, Do Not Invest: 0)
- Ensemble approach with multiple decision trees

Hyperparameters:

- Number of estimators (trees): To be optimized during training
- Maximum depth: Controlled to prevent overfitting
- Minimum samples split/leaf: Tuned for generalization
- Feature sampling: Random subset per tree for diversity
- Bootstrap sampling: Enabled for robust training

Training Strategy:

- Supervised learning with labeled historical data
- Train-test split for validation
- Cross-validation for hyperparameter tuning
- Feature importance analysis for model interpretability

AI Chatbot: Profile Classification & Portfolio Optimization

1. Investor Profiling Model (Machine Learning Classification)

The objective is to classify users into investment profiles based on input features (risk tolerance, capital, horizon, experience).

- Algorithm Type: Supervised learning classification
- Training Data: Synthetic investor dataset
- Process:
 1. Encode user features into numeric vectors
 2. Train model to map user attributes → risk profile
 3. Model outputs profile category (e.g., Conservative / Balanced / Aggressive)

The profiling step determines the portfolio strategy applied later in the recommendation pipeline.

2. Portfolio Optimization using HRP (Hierarchical Risk Parity)

A second allocation method is implemented using **HRP on 52 financial tickers**, providing a deterministic and risk-controlled portfolio alternative.

- **Input Data:** Historical price data for 52 selected assets
- **Steps:**
 1. Price → return computation
 2. Covariance & correlation matrix estimation
 3. Asset clustering through hierarchical tree
 4. Weight allocation proportional to risk contribution
- **Result:** A diversified allocation emphasizing **risk-balanced exposure across 52 tickers**, acting as a benchmark against RL results.

3. Reinforcement Learning Portfolio Allocation Agent

The RL agent learns an optimal portfolio allocation strategy using **~7 years of historical market data over 52 investment tickers**.

The objective is to maximize cumulative return while controlling volatility through a reward-based learning process.

- **Task:** Adaptive portfolio allocation over 52 assets
- **Training Approach:** Reinforcement Learning (interaction with market time-series)
- **Core Logic:**
 1. **State:** price variations, historical returns and market features
 2. **Action:** portfolio weight adjustment across 52 tickers
 3. **Reward:** portfolio performance over the episode (profit vs risk)
 4. **Policy Update:** iterative improvement based on cumulative reward
- **Outcome:** A trained policy capable of generating dynamic allocations instead of static rules.

Key Design Decisions:

- Integration of LLM for nuanced understanding of user psychology
- Multi-stage approach: classification → initial allocation → RL optimization
- Personalization based on individual risk profiles
- Continuous learning and adaptation through RL

Tools, Libraries, and Frameworks

Frontend Development:

- **Framework:** Next.js (React-based)
- **Language:** TypeScript
- **Styling:** Tailwind CSS
- **Authentication:** NextAuth (handles password hashing, session management)
- **State Management:** React hooks and context

Backend Development:

- **API Service 1:** FastAPI (Python) - for ML model serving
- **API Service 2:** NestJS (Node.js) - for business logic and data management

- **Database:** Supabase (PostgreSQL-based)
- **Authentication & Security:** NextAuth integration

Machine Learning & AI:

- **ML Library:** scikit-learn (Random Forest implementation)
- **Data Processing:** pandas, numpy
- **AI Integration:** OpenRouter API (LLM services)
- **RL Framework:** (To be specified - potentially Stable Baselines3 or custom implementation)

External Services:

- **Financial Data:** Finance API
- **Email Notifications:** SendGrid
- **Push Notifications:** WebPush API
- **Payment Processing:** PayPal, TunisieMonétique (Tunisian payment system)

Development Tools:

- **Version Control:** Git
- **API Testing:** Postman or similar
- **Deployment:** (To be specified based on cloud provider)

2.3. Implementation

System Design

Overall Architecture: MenaFina follows a microservices architecture with clear separation between frontend, backend services, and external integrations.

Architecture Components:

1. Frontend Layer (Next.js)

- Responsive user interface for all platform modules
- Client-side routing and state management
- Real-time updates and notifications
- Role-based UI rendering (Student/Professor/Admin)

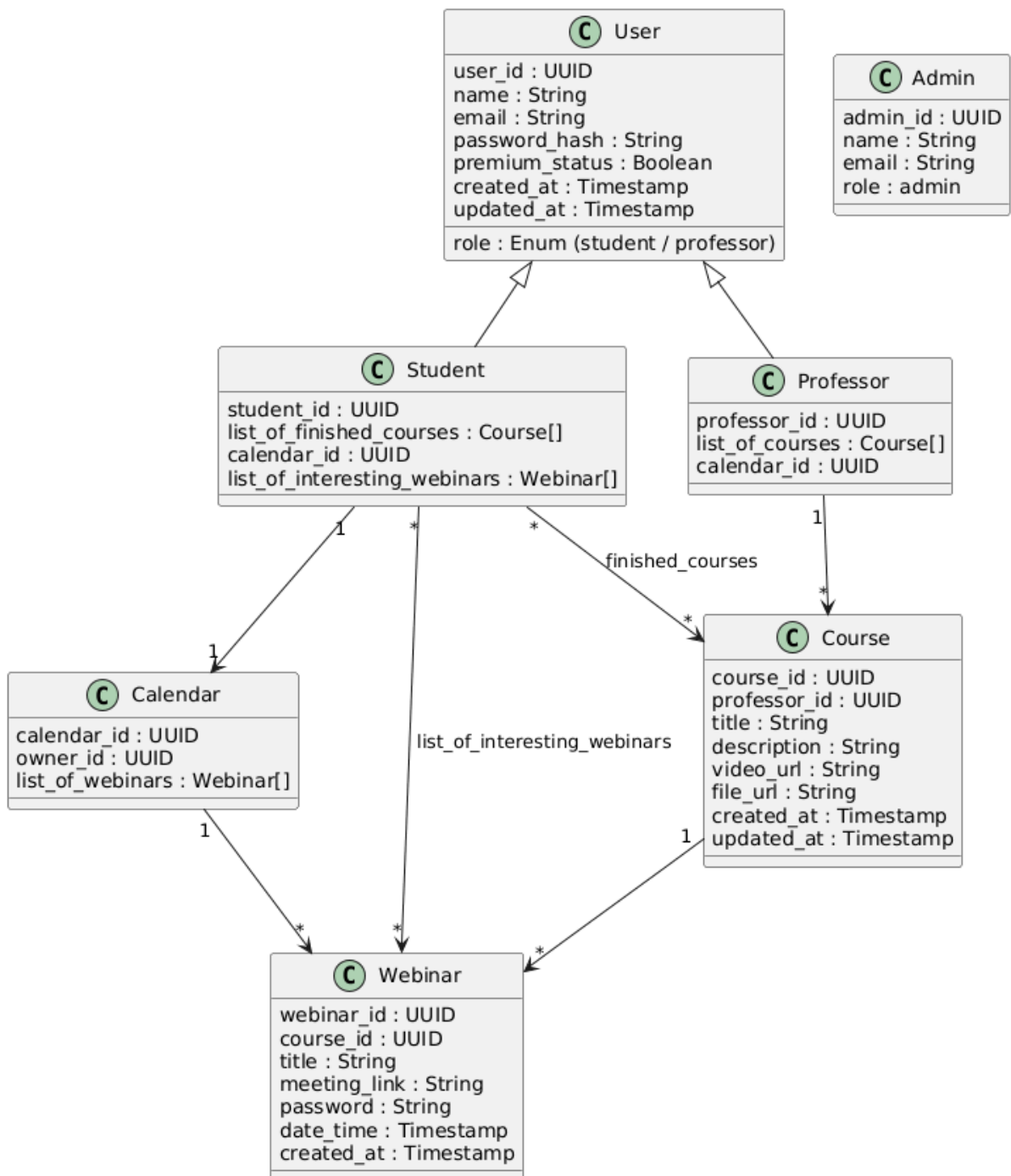
2. Backend Services Layer

FastAPI Service (Python):

- Handles real-time ML model inference
- Endpoints:
 - **/predict-investment:** Receives ticker symbol, fetches financial data, returns prediction
 - **/classify-profile:** Receives user questionnaire, returns entrepreneurial profile

- `/optimize-portfolio`: Receives portfolio data, returns RL-optimized allocation
 - Stateless design for horizontal scalability
 - Fast response times for real-time predictions
3. *NestJS Service (Node.js)*:
- Manages persistent data operations
 - RESTful API endpoints for:
 - User authentication and account management
 - Course CRUD operations
 - Webinar scheduling and enrollment
 - Rating and review system
 - Calendar synchronization
 - Statistics and analytics
 - Integration with Supabase for data persistence
 - Role-based access control (RBAC)
4. **Database Layer (Supabase)**
- PostgreSQL database for relational data
 - Tables: Users, Students, Professors, Courses, Webinars, Calendars, Ratings, Simulation History
 - Real-time subscriptions for live updates
 - Row-level security policies

MenaFina Planned Database ER Diagram



5. External Services Layer

- Finance API integration for market data
- OpenRouter API for LLM-powered chatbot
- SendGrid for email notifications
- WebPush API for browser notifications
- Payment gateways (PayPal, TunisieMonétique)

System Workflow & Pipeline:

Investment Simulator Workflow:

1. User enters company name/ticker in Simulator page (Frontend)
2. Frontend sends request to FastAPI endpoint `/predict-investment`
3. FastAPI calls Finance API to retrieve financial indicators
4. Financial data is preprocessed and fed into Random Forest model
5. Model outputs prediction: Invest (1) or Do Not Invest (0)
6. Result returned to Frontend and displayed to user
7. Simulation saved to history via NestJS → Supabase

AI Chatbot Workflow

The chatbot integrates user profiling, portfolio optimization and session tracking in a single interactive pipeline. It handles new and returning users differently, offers re-profiling options, and generates HRP and optionally RL portfolios.

1. User Entry

1. The chatbot starts by requesting the user's name.
2. `users.json` is loaded and checked:
 - If user **exists** → retrieve latest history entry
 - If user **does not exist** → a new profile is created later

2. Existing User Handling

If the user exists:

- Last stored session is displayed (profile, capital, date)
- The chatbot asks whether the user wants to **redo the profiling questionnaire**
 - If **yes** → complete questionnaire again → ML model predicts a new profile
 - If **no** → previous profile and capital are reused

This allows dynamic updating of investor profiles over time.

3. New User Handling

If no record is found:

- User completes full questionnaire (personality + financial behaviour + demographic)
- Inputs are preprocessed → feature vector is generated
- ML model predicts profile: conservative / balanced / aggressive
- A short explanation is provided

4. Portfolio Generation

Regardless of history:

HRP Portfolio (always executed first)

- `run_from_profile(profile,"hrp",capital)`
- Generates allocation across 52 tickers based on risk parity hierarchy

RL Portfolio (optional)

- User is asked: *"Lancer optimisation RL ? (y/n)"*
- If yes:
 - `run_rl()` uses HRP allocation as initial weights

- RL agent returns allocation in % and TND
- If *no*:
 - Session is saved with `rl_portfolio = null`

5. Saving Session to JSON

Each execution is stored using:

`save_user_profile(name, profile, capital, hrp_clean, rl_clean)`

Saved fields include:

- date
- profile
- capital
- hrp_portfolio (dict)
- rl_portfolio (percent + tnd + simulation result if available)

User history grows chronologically.

6. End Interaction

User may continue with new inputs or exit cleanly.

Professor-Student Space Workflow:

1. Professor creates course/webinar (Frontend)
2. Frontend sends data to NestJS API
3. NestJS stores course details in Supabase
4. Students browse courses (Frontend queries NestJS → Supabase)
5. Student marks webinar as "interesting" → added to their calendar
6. Notification triggered via WebPush API
7. Students rate courses → ratings stored in Supabase
8. Statistics aggregated and displayed on professor profiles

User Account Management Workflow:

1. User registers/logs in via NextAuth (Frontend)
2. Session managed by NextAuth with secure token handling
3. User data stored in Supabase via NestJS
4. Dashboard displays personalized data:
 - Profile information (Supabase query)
 - Calendar with upcoming webinars (Supabase + calendar sync)
 - Simulation history (Supabase query)
 - Course progress and ratings (Supabase query)

Integration Workflow Summary:

- **Frontend → FastAPI → ML Models → Frontend**
 - User inputs (company name or profile form) are sent via **FastAPI** to the appropriate Python model.
 - ML models process the data and return predictions, which are displayed immediately on the frontend.
- **Frontend → NestJS + Supabase → Frontend**

- Actions related to **accounts, courses, webinars, ratings, and statistics** are sent to NestJS, stored in Supabase, and retrieved as needed by the frontend.
- **Notifications & Premium Features**
 - Webinars and important events trigger notifications via **WebPush API**.

User Flows:

Student Flow:

- Homepage → Login/Register
- Navigate to:
 - Chatbot Page: Discover entrepreneurial profile, receive portfolio recommendations
 - Manage My Account: View personal data, statistics, calendar with webinars
 - Simulator Page: Enter ticker, receive investment prediction, view history
 - Professor–Student Space: Browse courses, filter content, visit professor profiles, mark webinars, rate courses

Professor Flow:

- All Student functionalities +
- Publish courses and tutorials
- Schedule webinars
- Restrictions: Cannot mark colleagues' webinars or rate any courses

Admin Flow (Future):

- Dedicated Admin Dashboard
- Monitor platform usage and popularity
- Oversee security and manage users
- Moderate content and review statistics

Programming Languages and Environments

Languages:

- **Frontend:** TypeScript (with React/Next.js)
- **Backend API 1:** Python (FastAPI)
- **Backend API 2:** TypeScript/JavaScript (NestJS)
- **Database Queries:** SQL (PostgreSQL via Supabase)
- **ML Models:** Python (scikit-learn, pandas, numpy)

Development Environments:

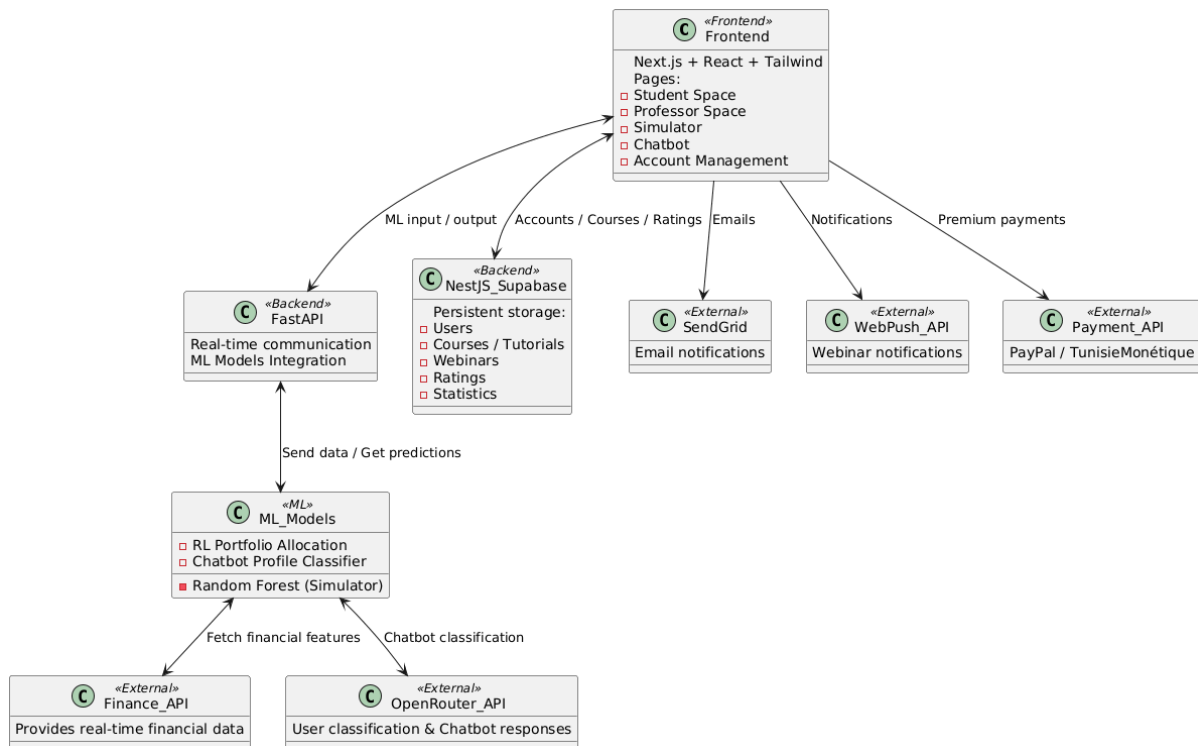
- **IDE:** VS Code or similar
- **Runtime:** Node.js (for Next.js and NestJS), Python 3.x (for FastAPI and ML)
- **Package Managers:** npm/yarn (JavaScript), pip (Python)

Deployment Environments:

- **Frontend:** Vercel or similar (Next.js optimized hosting)
- **Backend:** Cloud platform (AWS, GCP, Azure, or similar)
- **Database:** Supabase cloud hosting
- **Containerization:** Docker (for backend services)

Architecture Diagram

Global Architecture Overview:



Key Relationships:

- Professor → Course: One-to-Many
- Student → Calendar → Webinars: One-to-Many
- Student → Course: Many-to-Many (via finished courses)
- Course → Webinar: One-to-Many
- Student → Rating → Course: Many-to-Many

Privacy Note: The system does not track which specific student marked which webinar as interesting to protect user privacy.

3. Experiments & Results

3.1. Current Implementation Status

The MenaFina platform is in the **prototype and partial implementation phase**. The following components have been developed or are in progress:

Implemented Parts:

1. Frontend Prototype (Complete)

- Fully developed using Next.js, React, TypeScript, and Tailwind CSS
- Implements main user flows for Students and Professors
- Pages completed: Homepage, Chatbot, Simulator, Professor–Student Space, Account Management
- Responsive design with Tailwind CSS
- NextAuth integration for authentication flow

2. Investment Simulator ML Model (Partial)

- Random Forest model developed and trained
- Model ready to predict Invest/Do Not Invest decisions
- **Status:** Integration with frontend and FastAPI backend planned but not yet connected
- **Testing:** Model tested on historical data (specific metrics pending full evaluation)

3. AI Chatbot Services (Partial)

- Profile Classification service operational
- Portfolio Diversification algorithm implemented
- Reinforcement Learning optimizer architecture designed
- **Status:** Backend integration with frontend planned but not fully implemented
- **Testing:** Initial classification accuracy evaluated (specific metrics pending)

3.2. Experiments Conducted

Investment Simulator: Random Forest Model

Training Methodology:

- **Dataset:** Historical financial data from Finance API covering multiple companies across different sectors
- **Training Period:** Data spanning [specific timeframe to be documented]
- **Train-Test Split:** 80-20 split (80% training, 20% testing)
- **Cross-Validation:** K-fold cross-validation used for hyperparameter tuning

Hyperparameter Tuning:

- Grid search performed to optimize:
 - Number of estimators
 - Maximum tree depth
 - Minimum samples split/leaf
 - Feature sampling strategy

Evaluation Process:

1. Model trained on historical data with labeled investment outcomes
2. Predictions made on test set
3. Performance evaluated using multiple metrics
4. Feature importance analysis conducted

AI Chatbot: Profile Classification

Testing Methodology:

- **Test Data:** Synthetic user questionnaire responses representing various entrepreneurial profiles
- **Classification Categories:** [Number and types of profiles to be documented]
- **Evaluation:** Manual review of classification accuracy and relevance

Portfolio Optimization:

- **RL Training:** Reinforcement learning agent trained in simulated market environment
- **Reward Function:** Based on portfolio returns, risk metrics, and diversification
- **Evaluation:** Performance compared to baseline static allocation strategies

3.3. Metrics and Results

Investment Simulator Metrics (Preliminary)

The Random Forest model is evaluated using the following metrics:

Classification Metrics:

- **Accuracy:** Overall prediction correctness
- **Precision:** Proportion of correct "Invest" predictions
- **Recall:** Ability to identify all profitable investment opportunities
- **F1-Score:** Harmonic mean of precision and recall
- **ROC-AUC:** Model's ability to distinguish between classes

Financial Performance Metrics:

- **Prediction Latency:** Time from ticker input to prediction output
- **Feature Importance:** Top contributing features to investment decisions

Note: Specific numerical results pending full integration and testing phase.

AI Chatbot Metrics (Preliminary)

1. Profiling Model Tests

- Multiple user inputs were simulated through the questionnaire.
- The ML model successfully predicted profiles (*conservative / balanced / aggressive*).
- `explain_prediction()` confirmed feature influence and logic consistency.

2. HRP Portfolio Generation

- HRP allocation executed for different profiles and capital amounts.
- Allocation returned normalized weights over 52 tickers.
- Results stored correctly under `hrp_portfolio` in `users.json`.

3. RL Optimization

- RL module tested with optional user choice.
- When enabled, the system produced allocation in % and TND.
- If disabled, RL section remains null, confirming fallback behavior.

4. Session Storage & History Check

- Several user sessions created and reloaded (Amira, Malek, etc.).
- New users trigger full profiling; returning users may reuse or redo it.
- JSON log updates chronologically without overwriting previous sessions.

Frontend Performance Metrics

User Experience Metrics:

- **Page Load Time:** Average time to render key pages (Homepage, Simulator, Chatbot)
- **Responsiveness:** UI performance across desktop, tablet, mobile devices
- **Interaction Latency:** Time from user action to UI response

Accessibility:

- **NextAuth Authentication:** Successful login/registration rate
- **Navigation Flow:** User journey completion rates through key workflows

Note: Formal usability testing pending with target user groups.

3.4. Results Summary (Current Status)

Tables of Results :

Component	Status	Key Achievement	Pending Work
Frontend	✓ Complete	Fully functional UI for all modules	Backend integration
Simulator Model	• Partial	Model trained and ready+simulator ready interacting with clients via cli	FastAPI integration, live testing+Frontend link
AI Chatbot	• Partial	Profiling functional, HRP allocation working, RL optional + JSON history saved+terminal chatbot version complete ready for interactions	Frontend link + RL tuning+APIConnection
Backend (FastAPI)	✗ Planned	N/A	Full implementation
Backend (NestJS)	✗ Planned	N/A	Full implementation
Database (Supabase)	✗ Planned	N/A	Schema implementation
External APIs	• Partial	Finance API tested (yfinance OK)	Payment integration, notifications
Admin Module	✗ Planned	N/A	Complete development

Current Prototype Capabilities:

- ✓ Users can navigate through all frontend pages
- ✓ UI demonstrates complete user flows for Students and Professors
- ✓ ML models exist and function independently
- ✓ HRP Portfolio Allocation (52 tickers)
- ✓ Optional RL Portfolio Optimization
- ✓ Full Terminal-Based Workflow
- ✗ Real-time predictions not yet available through UI
- ✗ Data persistence not yet implemented
- ✗ User authentication functional but not connected to backend database

3.5. Comparison with Baseline/Alternative Approaches

Investment Prediction:

- **Random Forest vs. Simple Moving Average Strategy:** Random Forest expected to outperform due to incorporation of fundamental financial indicators beyond price trends
- **Random Forest vs. Linear Models:** Non-linear ensemble approach better captures complex market relationships

Portfolio Optimization:

- **RL-Based vs Static Allocation:** Reinforcement Learning provides *dynamic decision-making* instead of fixed allocation weights. It can theoretically adapt to changing markets rather than remaining constant over time.
- **RL vs Modern Portfolio Theory (MPT):** While MPT allocates based on historical variance/covariance, RL can *learn from reward outcomes and real market behavior*, allowing it to improve allocation decisions through experience rather than relying solely on historical correlations.

Note: Formal comparisons pending full system integration and live testing.

4. Discussion & Challenges

4.1. Limitations and Challenges

Technical Challenges

1. Backend Integration Complexity

- Challenge: Coordinating between two separate backend services (FastAPI for ML, NestJS for business logic) requires careful API design and synchronization.
- Impact: Increases development time and potential for integration bugs.
- Mitigation: Clear API documentation, standardized request/response formats, and comprehensive testing.

Note: The current prototype runs fully in terminal without backend/API connection. Backend integration will be a next step for deployment.

2. Real-Time ML Model Serving

- Challenge: Ensuring low-latency predictions (<2 seconds) for the Investment Simulator while fetching data from external Finance API and running Random Forest inference.
- Impact: Users experience degradation if response times are slow.
- Mitigation: Model optimization (pruning, quantization), caching frequently requested ticker data, asynchronous API calls.

In the chatbot prototype, inference is functional but remains dependent on yfinance data retrieval speed.

3. Data Quality and Availability

- Challenge: Finance API may have incomplete data for certain companies, especially smaller or international stocks.
- Impact: ML model receives incomplete features, potentially reducing prediction accuracy.
- Mitigation: Robust error handling, fallback strategies (use average values, skip prediction with warning), data validation pipelines.

HRP and RL allocation depend on external market data availability.

4. Reinforcement Learning Stability

- Challenge: RL agents can be unstable during training and may converge to suboptimal policies if reward function is poorly designed.
- Impact: Portfolio recommendations may be inconsistent or suboptimal.
- Mitigation: Careful reward shaping, extensive simulation testing, human oversight of recommendations.

In the current prototype, RL is optional and manually triggered, reducing risk but requiring future tuning.

5. Authentication and Security

- Challenge: Ensuring secure user authentication, protecting sensitive financial data, and preventing unauthorized access.
- Impact: Security breaches could compromise user trust and platform integrity.
- Mitigation: NextAuth for secure authentication, encrypted password storage, HTTPS for all communications, role-based access control.

Current prototype stores users in users.json without authentication. A security layer will be required for production.

Data-Related Challenges

6. Limited Historical Training Data

- Challenge: ML models perform best with large, diverse datasets. Limited access to comprehensive historical financial data may constrain model performance.
- Impact: Potential overfitting or reduced generalization to new companies/market conditions.
- Mitigation: Data augmentation, regularization techniques, cross-validation, continuous model retraining as more data becomes available.

7. Market Dynamics and Non-Stationarity

- Challenge: Financial markets are non-stationary; patterns that worked historically may not hold in the future.
- Impact: Model predictions may degrade over time as market conditions evolve.
- Mitigation: Regular model retraining, ensemble methods, incorporation of recent data, monitoring prediction performance over time.

8. Tunisian Market Specifics

- Challenge: Platform targets Tunisian users interested in foreign markets, but lacks integration with local Tunisian stock exchange data.

- Impact: Users cannot simulate investments in local companies.
 - Mitigation: Future integration with Tunisian financial data sources, USD-TND currency conversion module (planned).
-

User Experience Challenges

9. User Trust in AI Recommendations

- Challenge: Users may be skeptical of AI-driven investment advice, especially beginners unfamiliar with machine learning.
- Impact: Low adoption rates for chatbot and simulator features; users may ignore recommendations.
- Mitigation: Improve transparency, provide clearer explanations, add educational guidance.

Additionally, the chatbot currently operates via terminal, which limits user experience compared to a graphical interface.

5. Conclusion & Future Work

5.1. Summary

MenaFina is a specialized **financial education and investment intelligence platform** explicitly designed for the international market, focusing on democratizing access to complex investment knowledge. Our solution integrates two core AI modules:

1. **Investment Simulator: A Random Forest Classifier** analyzing over 60 factors for real-time, binary investment recommendations.
2. **AI Chatbot:** A conversational assistant that classifies the user's risk profile based on questionnaire inputs and generates portfolio recommendations using HRP, with an optional Reinforcement Learning (RL) agent for further optimization.

The platform's frontend (**Next.js/React**) and user flows (Student/Professor) are fully developed. The system is built on a **decoupled microservice architecture**, utilizing **FastAPI** for low-latency AI serving and **NestJS + Supabase** for persistent data management. While the models are validated, the final integration of the backend services with the live frontend is the project's current remaining development phase.

5.2. Main Challenges

The main challenge encountered during development was building the Investment Simulator using Yahoo Finance data. The raw API datasets contained inconsistent column names and missing financial metrics, which did not align with the expected input format for the model. To address this, we created two preprocessing functions—one dedicated to pricing data and another for fundamentals then a final merging function that combines, cleans, and formats the output into a model-ready feature vector. This step was essential for making the dataset usable. Beyond this technical issue, the project also faced broader challenges. We worked without real financial user data, relying on a synthetic dataset due to time and access limitations. The system was functional, but modules operated separately (Simulator ↔ Chatbot ↔ RL), which required careful handling of user history, interaction flow, and optional RL execution without breaking stability. Introducing reinforcement learning added additional sensitivity to reward tuning and convergence, making progressive iteration necessary. Despite these obstacles, each challenge pushed the system forward, leading to a stable prototype and establishing a solid foundation for future backend integration, real-data training, and model scalability.

5.3. What's Next

- **API Exposure**

Expose both the Investment Simulator and the AI Chatbot via FastAPI/NestJS to allow access from the web interface.

- **Database Migration**

Replace users.json with Supabase/PostgreSQL for multi-user management, persistent history, and scalable data storage.

- **Frontend Integration**

Connect the chatbot and simulator to the frontend for smooth web interaction, portfolio display, and user session tracking.

- **RL Optimization**

Improve the reinforcement learning agent (reward shaping, hyperparameter tuning, HRP vs RL comparison, backtesting scenarios).

- **Simulator Improvement**

Benchmark the Random Forest classifier, test alternative ML models, and increase accuracy on real market behaviour.

- **Dataset Evolution**

Currently based on synthetic data due to access limitations → future goal: acquire or collect real user/financial datasets to improve realism and model generalization.

- **Visualization & UX**

Add graphs & dashboards for portfolio evolution, performance comparison HRP/RL, and more educational explanations for users.

- **Strict/Flexible Mode Option (New Feature)**

Add a user-selectable mode: strict (conservative filtering) vs flexible (broader opportunities), influencing recommendation thresholds.

- **Self-Learning Simulator Loop (New Feature)**

Automatically update ticker_summary.csv using new_features.csv by adding a result

label (target = 1/0) after each simulation session.

If new data is detected → retrain model automatically;

If same company data is unchanged → skip retraining to avoid redundancy.

This allows continuous improvement over time and adaptive investment intelligence.